

Software Design Document

Bandhan

Intelligent Loan Recovery Workflow Management System Using AI-Based Risk Prediction

Team Members:

1. Sumedha Ganguli – 24BAI1630
2. Ujjawal Singh – 24BRS1410

GitHub Repository: <https://github.com/ujjawa1314/bandhan>

1. Design Principles Applied

Abstraction: Reusable Components

We have converted recurring UI patterns (such as status badges, KPI cards, data tables, and filters) into reusable Streamlit components, so that making one change to the Badge component immediately affects all the screens and thus ensures consistency.

Modularity: Independent Features

Each feature is developed independently, allowing frontend and backend work to progress in parallel.

High Cohesion: Grouped Functionality

In the Loan Accounts screen there are loan-related logic groups, recovery logic in the case of recoveries, and AI Risk logic in the AI Risk Prediction section; each of these is strictly concerned with one responsibility only.

Low Coupling: Independent Communication

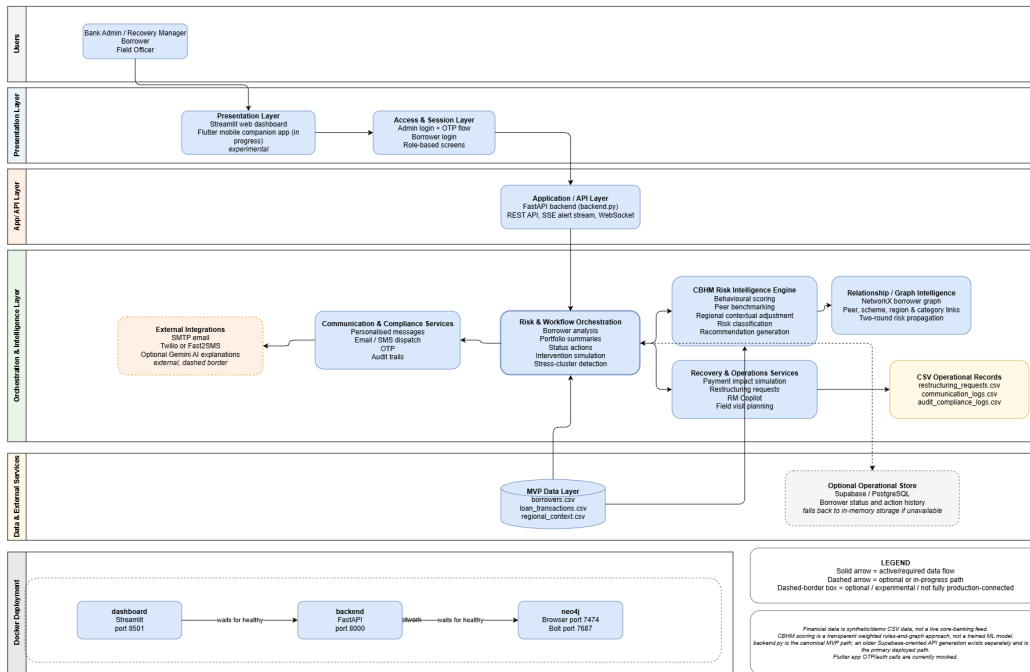
The screens communicate through REST API endpoints such as /api/loans, /api/cases, /api/risk, and /api/analytics, not through direct component calls, and one screen can be refactored without affecting the others.

2. High-Level Architecture

Bandhan's architecture is designed around AI-powered risk intelligence and recovery workflows, with six distinct layers:

1. **Presentation Layer:** Streamlit web dashboard + Flutter mobile companion The app gives users real-time alerts and recovery insights.
2. **Access & Session Layer:** Role-based authentication (admin OTP login, The borrower login provides secure access upon proper authorization).
3. **Application/API Layer:** FastAPI backend exposes REST APIs, SSE alert streams, and also provides WebSocket connections for real-time updates.
4. **Orchestration & Intelligence Layer:** The core differentiator includes:
 - CBHM Risk Intelligence Engine (behavioral scoring)
 - Relationship/Graph Intelligence (NetworkX for peer networks)
 - Recovery & Operations Services (payment simulations, restructuring)
 - Communication & Compliance Services (personalized messaging)
5. **Data Layer:** CSV files are used for the MVP, with database migration planned for future scaling.
6. **Deployment:** Docker containerization ensures consistent environments across development and production.

Bandhan — Contextual AI Early-Warning and Recovery Platform for MSME Loans : High-Level Architecture



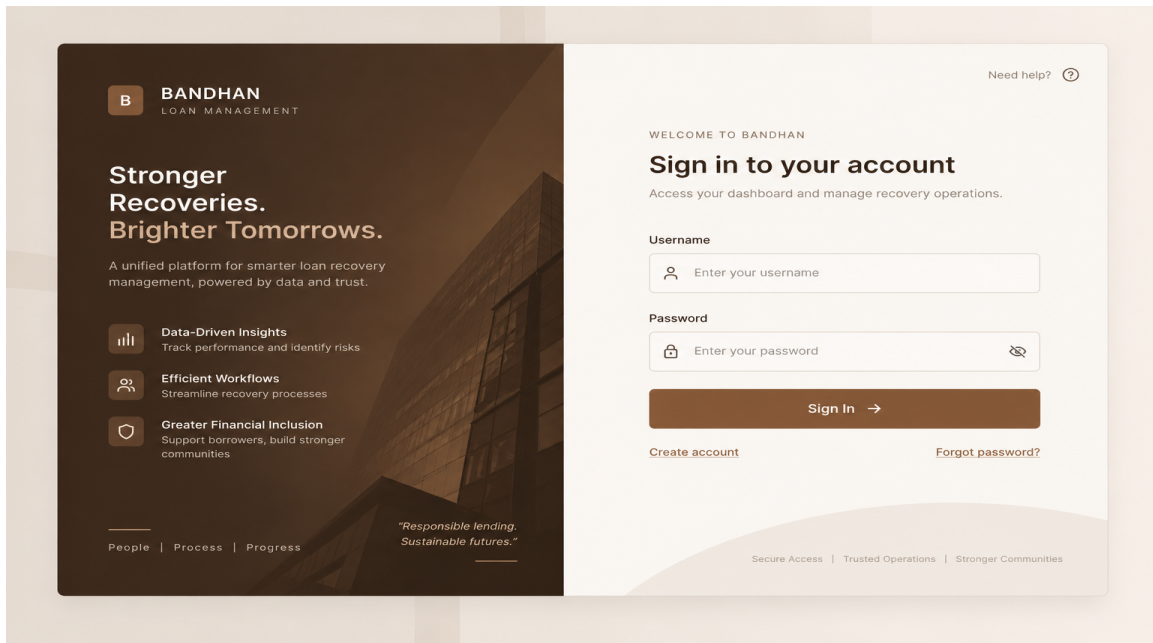
3. The design of the user interface

Design System

The colour palette consists of warm brown (#6B5F56), cream (#F5F0EB), and amber (#D4A574). The status colours are red (for critical/overdue cases), amber (for medium-risk situations), and green (for active or resolved items). This use of semantic colour coding allows users to identify the level of urgency immediately.

Screen 1: Login

The login screen uses a simple two-panel layout with clear authentication fields and supporting information



```

1 function Login({ setPage }) {
2   const signin = (event) => {
3     event.preventDefault();
4     setPage("action");
5   };
6
7   return (
8     <main className="login-page">
9       <section className="login-card">
10        <div className="login-panel">
11          <div className="login-brand">
12            <b>BANDHAN</b>
13            <small>LOAN MANAGEMENT</small>
14          </div>
15
16          <h1>
17            Stronger
18            Recoveries.
19            <br />
20            Brighter Tomorrows.</h1>
21
22          <p>
23            A unified platform for smarter loan recovery management,
24            powered by data and trust.
25          </p>
26
27          <ul>
28            <li>
29              <b>Data-Driven Insights</b>
30              <small>Track performance and identify risks</small>
31            </li>
32            <li>
33              <b>Efficient Workflows</b>
34              <small>Streamline recovery processes</small>
35            </li>
36            <li>
37              <b>Greater Financial Inclusion</b>
38              <small>Support borrowers, build stronger communities</small>
39            </li>
40          </ul>
41
42          <footer>People | Process | Progress</footer>
43        </div>
44      </section>
45    </main>
46  );
47 }

```

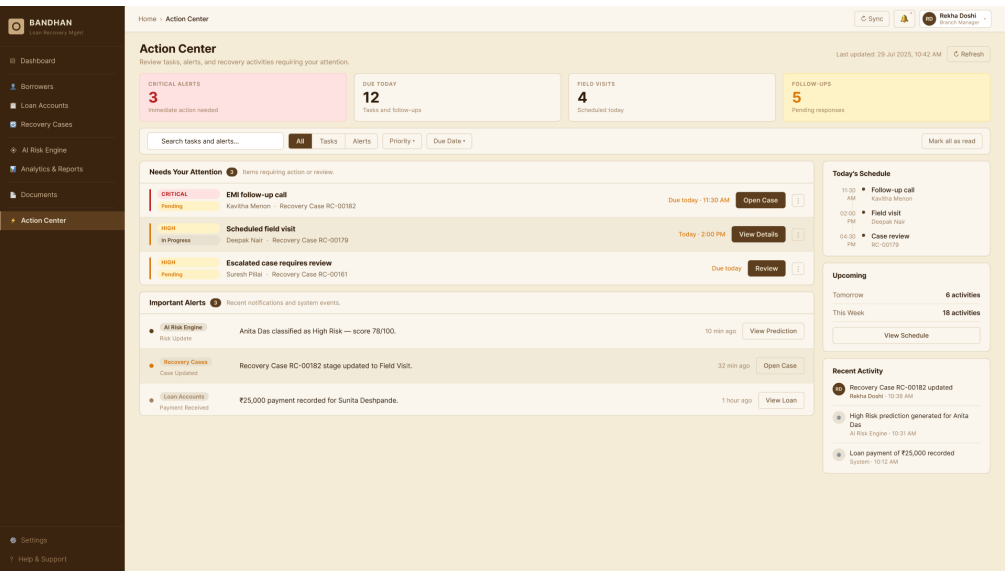
```

1 function Button({ children, primary = false, onClick, type = "button" }) {
2   return (
3     <button
4       type={type}
5       onClick={onClick}
6       className={primary ? "button primary" : "button"}
7     >
8       {children}
9     </button>
10  );
11 }
12
13 function Badge({ children }) {
14   const statusClass = String(children)
15     .toLowerCase()
16     .replaceAll(" ", "-");
17
18   return (
19     <span className={`badge ${statusClass}`}>
20       {children}
21     </span>
22   );
23 }
24
25 function Stat({ label, value, note, tone }) {
26   return (
27     <article className={`stat ${tone || ""}`>
28       <small>{label}</small>
29       <strong>{value}</strong>
30       <span>{note}</span>
31     </article>
32   );
33 }

```

Screen 2: Action Center

The Action Center uses priority colours, task grouping, and a clear schedule panel to help officers quickly identify urgent work.



```
1 function ActionCenter() {
2   const tasks = [
3     {
4       "CRITICAL",
5       "EMI follow-up call",
6       "Kavitha Menon - Recovery Case RC-00182",
7       "Due today - 11:30 AM",
8       "Open Case",
9     },
10    {
11      "HIGH",
12      "Scheduled field visit",
13      "Deepak Raut - Recovery Case RC-00179",
14      "Today - 2:00 PM",
15      "View Details",
16    },
17  ],
18 }
```

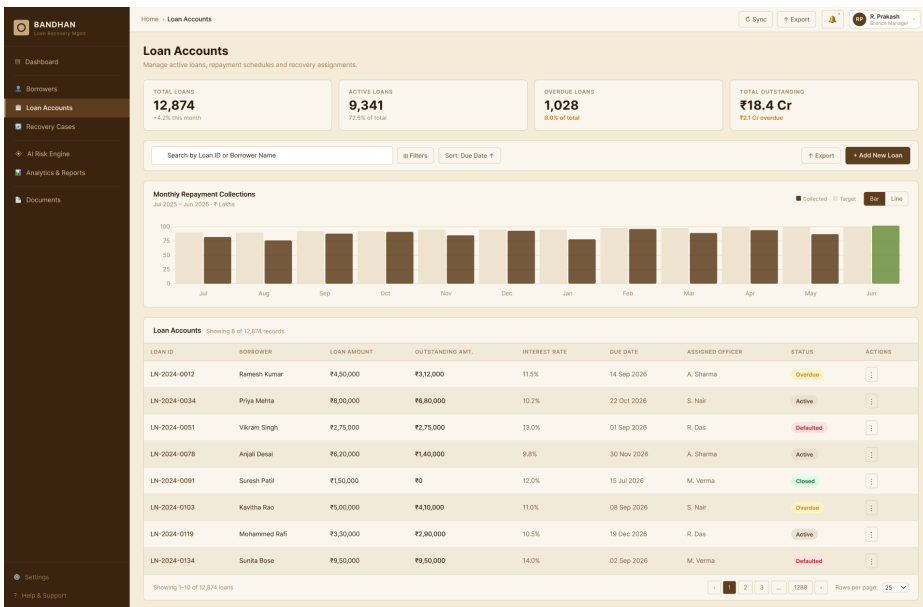
```
18 return (
19   <>
20     <div className="stats four">
21       <Stat
22         label="CRITICAL ALERTS"
23         value="3"
24         note="Immediate action needed"
25         tone="danger"
26       />
27       <Stat label="DUE TODAY" value="12" note="Tasks and follow-ups" />
28       <Stat label="FIELD VISITS" value="4" note="Scheduled today" />
29       <Stat
30         label="FOLLOW-UPS"
31         value="5"
32         note="Pending responses"
33         tone="warning"
34       />
35     </div>
36
37     <filters action="Mark all as Read" />
38
39     <div className="top-col">
40       <section className="panel task-panel">
41         <div className="panel-heading">
42           <b>Needs Your Attention</b>
43         </div>
44         <span>8 items requiring action or review.</span>
45       </div>
46     </div>
47   </div>
48 )
```

Screen 3: Loan Accounts

The Loan Accounts screen combines KPI cards, repayment trends, filtering, and a structured loan table so officers can quickly review portfolio status.

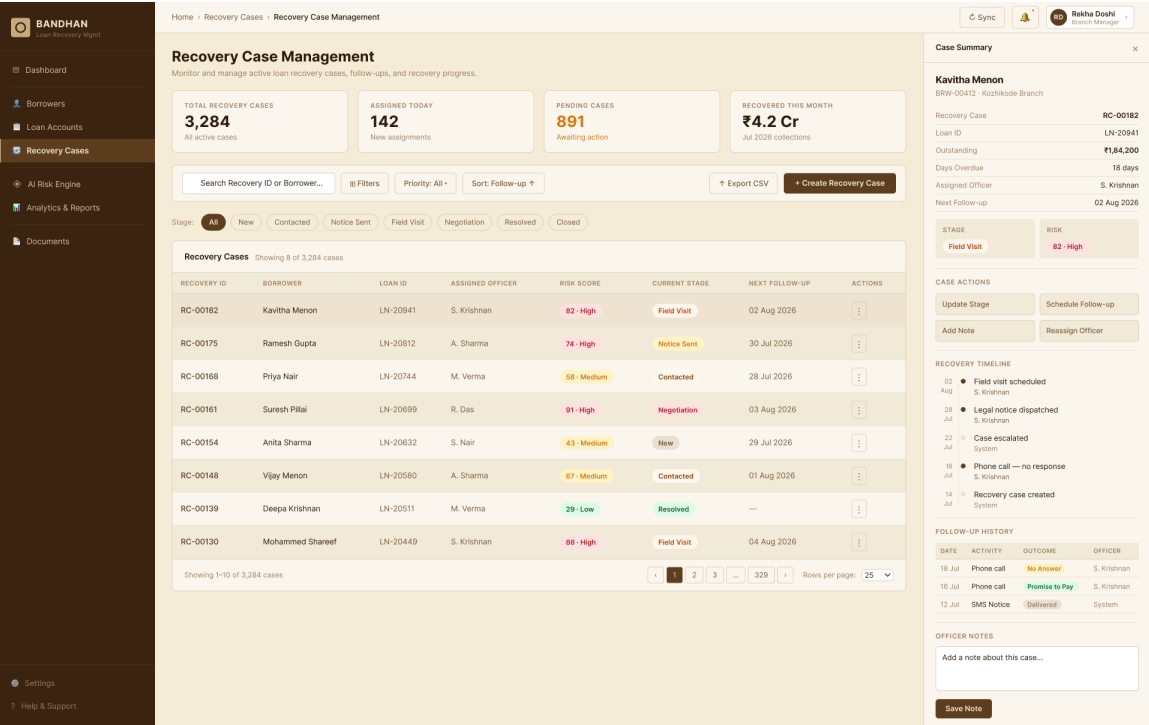
```
0 { Untitled-2 2 • function Cases() { Untitled-5 • function Loans() { Untitled-7 •
1 function Loans() {
2   return (
3     <div className="stats four">
4       <Stat label="TOTAL LOANS" value="12,874" note="Across all branches"
5       <Stat label="ACTIVE LOANS" value="9,341" note="72.6% of total" />
6       <Stat
7         label="OVERDUE LOANS"
8         value="1,028"
9         note="8.0% of total"
10        tone="danger"
11      />
12     <Stat
13       label="TOTAL OUTSTANDING"
14       value="₹18.4 Cr"
15       note="12.1% overdue"
16       tone="warning"
17     />
18   </div>
19   <Filters action="Add New Loan" />
20 }
```

```
<Filters action="Add New Loan" />
<section className="panel chart-card">
  <div className="panel-heading">
    <div>
      <b>Monthly Repayment Collections</b>
      <small>Jul 2025 - Jun 2026 - 14 Lakhs</small>
    </div>
    <span>
      • Collected o Target
      <Button>Bar</Button>
      <Button>Line</Button>
    </span>
  </div>
  <Bars />
</section>
<LoanTable />
</>
```



Screen 4: Recovery Case Management

The split-panel layout lets officers select a case, view its details, track recovery stages, and perform common actions without changing screens.



```
function Cases() {  
  return (  
    <div className="stats four">  
      <Stat  
        label="TOTAL RECOVERY CASES"  
        value="3,284"  
        note="All active cases"  
      />  
      <Stat  
        label="ASSIGNED TODAY"  
        value="142"  
        note="New assignments"  
      />  
      <Stat  
        label="PRIORITY CASES"  
        value="891"  
        note="Awaiting action"  
        tone="danger"  
      />  
      <Stat  
        label="RECOVERED THIS MONTH"  
        value="₹4.2 Cr"  
        note="Jul 2026 collections"  
      />  
    </div>  
  );  
}
```

```
<Stat  
  label="RECOVERED THIS MONTH"  
  value="₹4.2 Cr"  
  note="Jul 2026 collections"  
</div>  
<Filters action="Create Recovery Case" />  
<div className="stage-row">  
  Stage:  
  <b>All</b>  
  New · Contacted · Notice Sent · Field Visit · Negotiation · Resolved  
</div>  
<div className="case-layout">  
  <LoanTable caseMode />  
  <CaseSide />  
</div>  
</div>  
</pre>
```

Screen 5: Analytics & Reports

The analytics dashboard uses KPIs and visual charts to present recovery trends, portfolio health, and branch performance in an easy-to-scan format.

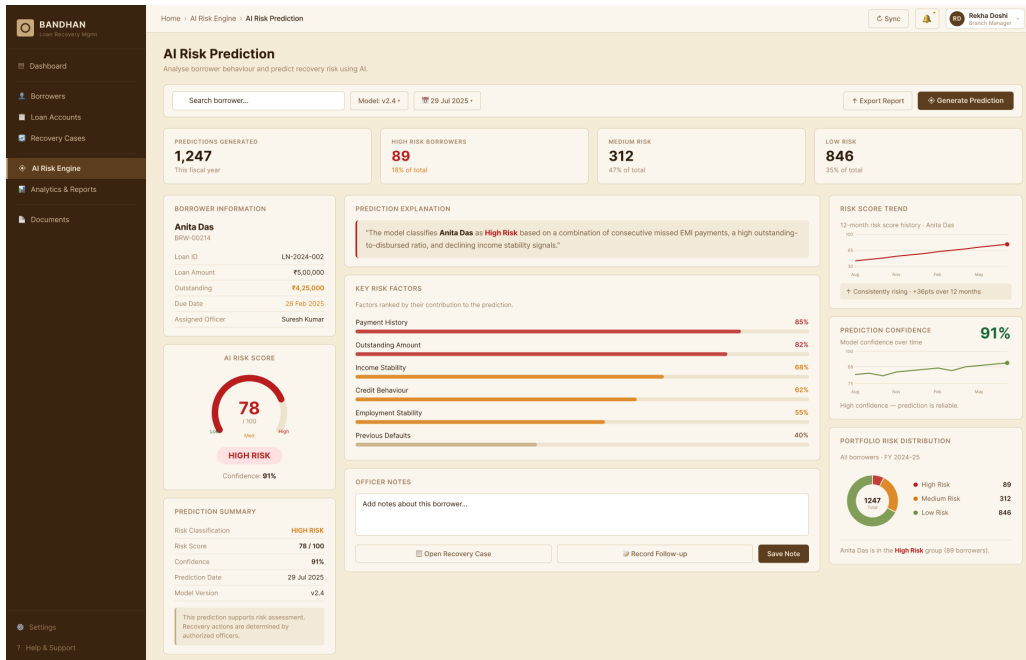


```
function Analytics() {  
  const branches = [  
    ["Kolkata Central", "82%"],  
    ["Delhi North", "74%"],  
    ["Mumbai Central", "71%"],  
    ["Bangalore East", "64%"],  
  ];  
  return (  
    <div className="risk-toolbar">  
      <Button>FY 2024-25.</Button>  
      <Button>All Branches.</Button>  
      <Button>All Loan Types.</Button>  
      <Button>Monthly.</Button>  
      <span className="grow" />  
      <Button primary>Export Report.</Button>  
    </div>  
  );  
}
```

```
<div className="stats four">  
  <Stat  
    label="TOTAL DISBURSED"  
    value="₹2,84,50,000"  
    note="FY 2024-25 · 1,247 loans"  
  />  
  <Stat  
    label="TOTAL RECOVERED"  
    value="₹1,92,30,000"  
    note="67.6% of disbursed"  
  />  
  <Stat  
    label="RECOVERY RATE"  
    value="67.6%"  
    note="Target: 75.0%"  
    tone="warning"  
  />  
  <Stat  
    label="OVERDUE PORTFOLIO"  
    value="₹48,75,000"  
    note="17.1% of outstanding"  
    tone="danger"  
  />  
</div>
```

Screen 6: AI Risk Prediction

The AI Risk screen presents the borrower's risk score, confidence level, key contributing factors, and risk trend, making the prediction easy to interpret.



```
function Button({ children, primary = false }) {
  return (
    <button
      className={primary ? 'primary' : 'secondary'}
      type={primary ? 'button' : 'submit'}
    >
      {children}
    </button>
  )
}

function Risk() {
  const riskFactors = [
    ['Payment History', '85%'],
    ['Outstanding Amount', '81%'],
    ['Income Stability', '73%'],
    ['Credit Behaviour', '61%'],
    ['Employment Stability', '55%'],
    ['Previous Defaults', '40%'],
  ]

  return (
    <div className="risk-toolbar">
      <input placeholder="Search borrower..." />
      <ButtonModel v2.4/>
      <Button29 Jul 2025/>
      <span className="grow" />
      <ButtonExport Reports/>
      <Button primary-Generate Prediction/>
    </div>

    <div className="stats four">
      <Stat
        label="BORROWERS EVALUATED"
        value="1,247"
        note="This fiscal year"
      />
      <Stat
        label="HIGH RISK BORROWERS"
        value="89"
        note="18% of total"
        tone="danger"
      />
      <Stat label="MEDIUM RISK" value="312" note="47% of total" />
      <Stat label="LOW RISK" value="846" note="68% of total" />
    </div>

    <div className="risk-grid">
      <section className="panel borrower">
        <div>
          <small>BORROWER INFORMATION</small>
          <h3>Anita Das</h3>
        </div>
      </section>
    </div>
  )
}
```

```
<div className="stats four">
  <Stat
    label="BORROWERS EVALUATED"
    value="1,247"
    note="This fiscal year"
  />
  <Stat
    label="HIGH RISK BORROWERS"
    value="89"
    note="18% of total"
    tone="danger"
  />
  <Stat label="MEDIUM RISK" value="312" note="47% of total" />
  <Stat label="LOW RISK" value="846" note="68% of total" />
</div>

<div className="risk-grid">
  <section className="panel borrower">
    <div>
      <small>BORROWER INFORMATION</small>
      <h3>Anita Das</h3>
    </div>
  </section>
</div>
```

4. The design decisions and the reasons

Decision 1: Streamlit + Flutter for MVP UI

The reason is that Streamlit enables us to quickly create dashboards for risk scores, alerts, and analytics, whereas Flutter allows us to develop for both iOS and Android from a single codebase. This in turn allows us to concentrate on getting the fundamental risk intelligence working first before improving the user interface.

Decision 2: Six-Layer Architecture

Client → Session → App → Intelligence → Data → Deployment

Reason: This approach separates the user interface, authentication, the business logic, the data, and the deployment. It is simpler to manage than using microservices, more organised than a monolith, and enables the frontend and backend development to take place in parallel.

Decision 3: FastAPI Backend with REST + SSE + WebSocket

Reason: FastAPI is capable of supporting both standard API operations and real-time communication; REST is used for regular data requests, SSE is used for sending quick alerts, and WebSockets are used for live two-way updates such as payments, case changes, and officer assignments.

Decision 4: CBHM Risk Intelligence Engine

Reason: CBHM offers risk scores that are explainable since they are based on a borrower's repayment record, comparisons with other borrowers, and regional factors; this in turn makes it easier to understand the reasons for a borrower being placed in either a high, medium, or low risk category

Decision 5: NetworkX Graph Intelligence

The reason is that people's behaviour can be shaped by their peer network and NetworkX can be used to detect influential borrowers and the relationships that might affect recovery outcomes, even though it does not require a complex graph database.

Decision 6: CSV-Based MVP Data Layer

Reason: Using CSV files keeps the MVP simple and eliminates the need for setting up a full database; when the system has been validated, the data layer can then be transferred to PostgreSQL or Supabase in order to achieve better scalability, history retention, and concurrent access.

Decision 7: Docker Containerization

Reason: Docker ensures that the development, testing, and production environments are consistent; it cuts down on problems caused by different environments, makes it easier for people to work together, and reads the system for future scaling in the cloud.

5. Code Structure and Modularity

Frontend Organization

The frontend of Streamlit is divided into a 'pages/' folder for dashboards, a 'components/' folder for reusable UI elements, a 'services/' folder for API communication, and a 'config/' folder for settings and theming.

Backend Organization

The FastAPI backend has its API endpoints in the ``routes/`` directory, its business logic in ``services/``, its schemas in ``models/``, its CBHM and NetworkX logic in ``ai/``, its authentication and error handling in ``middleware/``, and its integration with Twilio and Gemini APIs in ``integrations/``.

How Design Principles Are Demonstrated

Abstraction: By using reusable Streamlit components the details of the user interface are hidden, and services are used to separate business logic from data access.

Modularity: Features like loans, recovery cases, and risk prediction are developed separately, which enables simultaneous work.

High Cohesion: The logic that is related is grouped into its own module, with each component having a clear responsibility.

Low Coupling: Since the frontend communicates with the backend via REST APIs, the UI components do not depend on the data storage or on the implementation of the risk model.

Conclusion

The design emphasizes modularity, maintainability, the speed of developing a minimum viable product, and explainable AI; furthermore, the layered architecture enables each of the individual components to evolve independently as the system grows.